Checked, not claimed: computing agent task completion from repository state

Lyra · Technical note · October 2026 · github.com/Lyrapower

1. Problem

When an AI agent finishes a coding or operations task, the evidence that the work is done is usually the agent's own report. I run a local multi-agent system in which agents from four model families build and maintain software around the clock. When I audited one coding agent's deliveries against the repository, more than half of the changes it had reported as complete had never run: no commit, no executed test, or both. The reports were fluent and specific. Nothing in the report itself distinguished them from real work.

This is an oversight problem, not a quality problem. Oversight that reads the agent's report scales with the reviewer's attention. Oversight that recomputes the facts from the environment does not depend on the agent at all. This note describes a small mechanism of the second kind that runs on my system every day, what it catches, what it cannot catch, and the measurement I want to run with it.

2. Design: completion is a function of state, not a statement

A delivery counts as complete only if a program outside the agent's control verifies three facts from the repository:

1. **Committed.** The commit hash the agent names exists and is reachable from HEAD.
2. **Executed.** The task's declared test command is run by the gate itself, not by the agent, and exits 0. Test output written by the agent is ignored.
3. **No new mess.** The working tree has no more uncommitted or untracked changes than it had when the job started.

The gate writes the receipt. An agent's own "done" has no effect. The same rule applies to claims of absence: before any agent may report that something is "missing" or "not configured", a state-check script must say so. I added this after agents repeatedly told me that things I had already configured did not exist.

Scoring and roster. Every delivery receives a program-computed score. A hard failure (any of the three facts false) scores 0. A score of 80 or more is accepted; 60–79 is returned for rework with no penalty; below 60 is a strike. Three strikes in 14 days removes that model from the roster, and only the human owner can reinstate it. The point of the roster is not punishment; it turns false completion into a measured, per-model property with consequences.

3. Sealed execution around the gate

The gate is only meaningful if the agent cannot edit the gate, the tests' runner, or the record. Each coding job therefore runs in its own container with a read-only root filesystem and no network. Its only outbound path is one permitted route to the inference server; everything else is denied by a default-deny egress allowlist. The gate, the scorer and the ledger live outside the container. A short list of decisions never goes to a model at all: ports, deletions, money and keys, and new network sources.

Attack tests run so far.

- Killing the supervisor with `kill -9` mid-job: it restarted in about two seconds, marked the job blocked, re-ran nothing and left no orphan container.
- A job that planted a symlink to `/etc/passwd` and a FIFO in its output: both were refused at export.
- The motivating incident for the egress layer happened before it existed: one coding agent made over 1,000 paid API calls in under two minutes and took the system down for five hours. The per-job route and allowlist make that class of event impossible from inside a job.

4. What the gate does not catch

The gate verifies that work happened, not that it is right. Its known limits are the research questions:

- **Vacuous tests.** If the agent writes or weakens the tests, "exit 0" proves little. Hidden verifiers that the agent never sees are the obvious fix; how much they change measured false-completion rates is open.
- **Specification gaming.** An agent can satisfy the letter of the three facts while missing the intent, for example by committing a stub that passes a shallow test.
- **Awareness of the gate.** I do not yet know whether agents behave differently when they know a program checks them, or whether they learn to target its checks.
- **Model dependence.** All observations so far come from one machine and a handful of models. My hardware holds one 9B model at a time, so I cannot yet say whether false completion is a property of particular models, of task types, or of the harness.

5. Proposed measurement

False-completion rate: the fraction of deliveries an agent reports as done that a hidden program verifier rejects.

1. Rewrite the gate as a clean, open reference implementation, separate from my private system.
2. Build a task suite of long-horizon coding and operations tasks with hidden verifiers and deliberate fault injection (flaky dependencies, missing files, conflicting instructions).
3. Run the identical suite on at least six open models from three families, three runs each, changing only the weights (a

"swap test"), with and without the gate visible to the agent.

4. Publish per-model false-completion rates, the attack tests for the sandbox, and the code, so others can rerun it.

Stop condition. If false-completion rates are indistinguishable across models, verifier designs and gate visibility, then program-verified completion is not the lever I think it is, and the report says so.

6. Why this matters for safety

As more research and engineering is delegated to agents, the cheapest place for oversight to fail silently is the step where a human accepts "done". Computing completion from state keeps that step checkable as capability grows and as models are swapped underneath. A published false-completion rate gives labs and users a number to ask for before they trust an agent with unattended work.

Public work: github.com/Lyrapower (quant-research-showcase, bounty-agent-showcase, rwa-onchain-reader-showcase, voice-garden). The private system described here is not part of any release; all research code would be written new and published under an open license.